Лабораторна робота №2

Оптимізація багатоекстремальної функції в дискретному просторі з обмеженнями з використанням алгоритму імітації відпалу

Мета: Вивчення стохастичних методів оптимізації, практичне освоєння алгоритму імітації відпалу та дослідження його ефективності під час пошуку прихованих глобальних екстремумів в умовах статичних просторових обмежень.

Варіант №3

1. Опис алгоритму

Фактично, програма - це шукач, котрий бродить картою, аби знайти сховані об’єкти. Коли він стає на клітинку, йому кажуть наскільки сильний тут сигнал. Що ближче до об’єкта - тим сигнал сильніший, а його ціль - знайти місце з найсильнішим сигналом. Тож щоб не топтатися на одному місці, програма використовує метод відпалу. Це працює так: на початку він дуже активний (ніби гарячий) і може робити випадкові кроки, навіть якщо там сигнал гірший, оскільки це допомагає йому не застрягти біля якоїсь випадкової точки сигналу. Але з часом він охолоджується, стає більш обережним і рухається тільки туди, де сигнал стає кращим, щоб точно знайти ціль. Коли об’єкт знайдено - він зникає з карти. Але програма вже охолола, тож аби знайти наступний об’єкт, він знову нагрівається, тобто він має змогу знову робити великі випадкові кроки. Так продовжується, поки він не знайде всі об’єкти.

2. Реалізація.

Завдання 1. Реалізувати клієнтську програм, що підключається до сервера (*ooi-lab2.up.railway.app*) лабораторної роботи за наданим API.

```
BASE_URL = "https://ooi-lab2.up.railway.app"
STUDENT_NAME = "Бубон Л. В."
VARIANT_NUMBER = 3
```

```
def start_session():
    response = requests.post(
        BASE_URL + "/api/start",
        json={"student_name": STUDENT_NAME, "variant_number": VARIANT_NUMBER},
        timeout=15,
    )
    response.raise_for_status()
    data = response.json()
    start = (data["start_position"]["x"], data["start_position"]["y"])
    return data["session_id"], start, data["map"]


def query_cell(session_id, x, y):
    response = requests.post(
        BASE_URL + "/api/move",
        json={"session_id": session_id, "target_position": {"x": x, "y": y}},
        timeout=15,
    )
    response.raise_for_status()
    return response.json()
```

Завдання 2. Підібрати гіперпараметри алгоритму імітації відпалу (початкова температура T_{start}, закон охолодження, функція залежності радіуса R(T))

```
T_START = 30.0
COOLING_RATE = 0.98
T_MIN = 0.3
R_MAX = 25
STAGNATION_LIMIT = 80

def temperature(k):
    return max(T_MIN, T_START * (COOLING_RATE ** k))

def radius(current_temperature):
    return max(1, round(R_MAX * current_temperature / T_START))
```

Параметр	Значення	Призначення
T_START	30.0	початкова температура
COOLING_RATE (α)	0.98	коефіцієнт геометричного охолодження
T_MIN	0.3	нижня межа температури
R_MAX	25	максимальний радіус клітинок
STAGNATION_LIMIT	80	к-сть ітерацій без покращень до повторного нагрівання
MAX_SERVER_STEPS	20000	ліміт запитів до сервера

Завдання 3. Домогтися успішного знаходження всіх прихованих об’єктів на карті свого варіанта

```
def energy(signal_level):
    return -signal_level

def neighbor(position, grid, r):
    height = len(grid)
    width = len(grid[0])
    x0, y0 = position
```

```
    for _ in range(60):
        x = x0 + random.randint(-r, r)
        y = y0 + random.randint(-r, r)
        if (x, y) == (x0, y0):
            continue
        if x < 0 or y < 0 or x >= width or y >= height:
            continue
        if grid[y][x] == 1:
            continue
        return (x, y)
    return None

def acceptance_probability(delta_energy, current_temperature):
    if delta_energy < 0:
        return 1.0
    return math.exp(-delta_energy / current_temperature)

        while iteration < MAX_ITERATIONS and steps_made < MAX_SERVER_STEPS:
    if remaining == 0 or viz.closed:
        break
    iteration += 1
    current_temperature = temperature(k)
    temperature_history.append(current_temperature)
    r = radius(current_temperature)
    candidate = neighbor(current, grid, r)
    if candidate is None:
        k += 1
        continue
    if candidate in cache:
        candidate_signal = cache[candidate]
    else:
        response = query_cell(session_id, candidate[0], candidate[1])
        candidate_signal = response["signal_level"]
        steps_made = response["steps_made"]
        remaining = response["remaining_foxes"]
        last_signal = candidate_signal
        if response["fox_caught"]:
            found.append(candidate)
            print(f"Знайдено об'єкт #{len(found)} у {candidate}, "
                  f"кроків: {steps_made}, залишилось: {remaining}")
            cache.clear()
            current = candidate
            path.append(current)
            current_energy = math.inf
            best_energy = math.inf
            k = 0
            no_improve = 0
            viz.update(path, found,
                       f"Кроків: {steps_made} | Залишилось: {remaining} | "
                       f"T: {T_START:.1f} (reheat)")
            continue
        cache[candidate] = candidate_signal
        if steps_made % SHOW_EVERY == 0:
            viz.update(path, found,
                       f"Кроків: {steps_made} | Залишилось: {remaining} | "
                      f"Сигнал: {last_signal:.2f} | T: {current_temperature:.2f}")
    candidate_energy = energy(candidate_signal)
    delta_energy = candidate_energy - current_energy
    probability = acceptance_probability(delta_energy, current_temperature)

    if random.random() < probability:
        current = candidate
        current_energy = candidate_energy
        path.append(current)
```

```
    if candidate_energy < best_energy - 1e-9:
        best_energy = candidate_energy
        no_improve = 0
    else:
        no_improve += 1
        if no_improve >= STAGNATION_LIMIT:
            k = 0
            no_improve = 0
            continue
    k += 1
```

Завдання 4. Реалізувати візуалізацію руху агента в реальному часі. На карті мають відображатися перешкоди, робот і постійні позначки місць, де об'єкти було успішно знайдено.

```
   def update(self, path, found, title):
    pg = self.pg
    for event in pg.event.get():
        if event.type == pg.QUIT:
            self.closed = True

    while self.drawn < len(path) - 1:
        pg.draw.line(self.path_surface, (30, 100, 220, 150),
                     self._center(path[self.drawn]),
                     self._center(path[self.drawn + 1]), 1)
        self.drawn += 1
    self.screen.blit(self.map_surface, (0, 0))
    self.screen.blit(self.path_surface, (0, 0))

    for p in found:
        c = self._center(p)
        pg.draw.circle(self.screen, (0, 0, 0), c, self.CELL + 2)
        pg.draw.circle(self.screen, (255, 215, 0), c, self.CELL)
    rx, ry = self._center(path[-1])
    pg.draw.rect(self.screen, (220, 20, 20),
                 (rx - self.CELL // 2, ry - self.CELL // 2,
                  self.CELL, self.CELL))
    bar_top = self.height * self.CELL
    pg.draw.rect(self.screen, (20, 20, 20),
                 (0, bar_top, self.width * self.CELL, self.BAR))
    text = self.font.render(title, True, (255, 255, 255))
    self.screen.blit(text, (8, bar_top + 7))
    pg.display.flip()
```

Повний код:

```
import math
import random
import subprocess
import sys
import requests
import matplotlib
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

BASE_URL = "https://ooi-lab2.up.railway.app"

STUDENT_NAME = "Бубон Л. В."
VARIANT_NUMBER = 3
```

```
T_START = 30.0
COOLING_RATE = 0.98
T_MIN = 0.3
R_MAX = 25
STAGNATION_LIMIT = 80

MAX_SERVER_STEPS = 20000
MAX_ITERATIONS = 300000

SHOW_EVERY = 3
KEEP_WINDOW_OPEN = True
RANDOM_SEED = None

RUN_EXPERIMENTS = False //True, аби запустити перевірку
RUNS_PER_EXPERIMENT = 3
EXPERIMENTS = [(0.90, 30.0), (0.95, 30.0), (0.98, 30.0), (0.99, 30.0), (0.98,
10.0), (0.98, 50.0)]

def start_session():
    response = requests.post(
        BASE_URL + "/api/start",
        json={"student_name": STUDENT_NAME, "variant_number": VARIANT_NUMBER},
        timeout=15,
    )
    response.raise_for_status()
    data = response.json()
    start = (data["start_position"]["x"], data["start_position"]["y"])
    return data["session_id"], start, data["map"]


def query_cell(session_id, x, y):
    response = requests.post(
        BASE_URL + "/api/move",
        json={"session_id": session_id, "target_position": {"x": x, "y": y}},
        timeout=15,
    )
    response.raise_for_status()
    return response.json()

def energy(signal_level):
    return -signal_level

def temperature(k):
    return max(T_MIN, T_START * (COOLING_RATE ** k))

def radius(current_temperature):
    return max(1, round(R_MAX * current_temperature / T_START))

def neighbor(position, grid, r):
    height = len(grid)
    width = len(grid[0])
    x0, y0 = position

    for _ in range(60):
        x = x0 + random.randint(-r, r)
        y = y0 + random.randint(-r, r)
        if (x, y) == (x0, y0):
            continue
        if x < 0 or y < 0 or x >= width or y >= height:
            continue
        if grid[y][x] == 1:
            continue
        return (x, y)
    return None
```

```
def acceptance_probability(delta_energy, current_temperature):
    if delta_energy < 0:
        return 1.0
    return math.exp(-delta_energy / current_temperature)

class PygameVisualizer:
    CELL = 8
    BAR = 34

    def __init__(self, grid, start):
        import pygame
        self.pg = pygame
        pygame.init()
        self.height = len(grid)
        self.width = len(grid[0])
        map_w = self.width * self.CELL
        map_h = self.height * self.CELL
        self.screen = pygame.display.set_mode((map_w, map_h + self.BAR))
        pygame.display.set_caption(f"Lab2_Bubon - Варіант {VARIANT_NUMBER}")

        self.font =
pygame.font.SysFont("dejavusans,arial,liberationsans,freesans", 18)
        self.map_surface = pygame.Surface((map_w, map_h))
        self.map_surface.fill((240, 240, 240))
        for y in range(self.height):
            for x in range(self.width):
                if grid[y][x] == 1:
                    pygame.draw.rect(self.map_surface, (46, 125, 50),
                                     (x * self.CELL, y * self.CELL,
                                      self.CELL, self.CELL))
        self.path_surface = pygame.Surface((map_w, map_h), pygame.SRCALPHA)
        self.drawn = 0
        self.closed = False
        self.clock = pygame.time.Clock()

    def _center(self, p):
        return (p[0] * self.CELL + self.CELL // 2,
                p[1] * self.CELL + self.CELL // 2)

    def update(self, path, found, title):
        pg = self.pg
        for event in pg.event.get():
            if event.type == pg.QUIT:
                self.closed = True

        while self.drawn < len(path) - 1:
            pg.draw.line(self.path_surface, (30, 100, 220, 150),
                         self._center(path[self.drawn]),
                         self._center(path[self.drawn + 1]), 1)
            self.drawn += 1
        self.screen.blit(self.map_surface, (0, 0))
        self.screen.blit(self.path_surface, (0, 0))

        for p in found:
            c = self._center(p)
            pg.draw.circle(self.screen, (0, 0, 0), c, self.CELL + 2)
            pg.draw.circle(self.screen, (255, 215, 0), c, self.CELL)
        rx, ry = self._center(path[-1])
        pg.draw.rect(self.screen, (220, 20, 20),
                     (rx - self.CELL // 2, ry - self.CELL // 2,
                      self.CELL, self.CELL))
        bar_top = self.height * self.CELL
        pg.draw.rect(self.screen, (20, 20, 20),
                     (0, bar_top, self.width * self.CELL, self.BAR))
```

```
        text = self.font.render(title, True, (255, 255, 255))
        self.screen.blit(text, (8, bar_top + 7))
        pg.display.flip()

    def save(self, filename):
        self.pg.image.save(self.screen, filename)

    def wait_close(self):
        pg = self.pg
        print("Закрийт вікно візуалізації, щоб завершити.")
        while not self.closed:
            for event in pg.event.get():
                if event.type in (pg.QUIT, pg.KEYDOWN):
                    self.closed = True
            self.clock.tick(30)
        pg.quit()

class MatplotlibVisualizer:
    def __init__(self, grid, start):
        self.closed = False
        plt.ion()
        self.fig, self.ax = plt.subplots(figsize=(8, 8))
        cmap = ListedColormap(["#f0f0f0", "#2e7d32"])
        self.ax.imshow(grid, cmap=cmap, origin="upper", vmin=0, vmax=1)
        self.ax.set_xticks([])
        self.ax.set_yticks([])
        (self.path_line,) = self.ax.plot([], [], color="tab:blue",
                                         linewidth=0.6, alpha=0.6)
        (self.found_marks,) = self.ax.plot([], [], marker="*", color="gold",
                                           markeredgecolor="black",
                                           markersize=16, linestyle="")
        (self.robot,) = self.ax.plot([start[0]], [start[1]], marker="o",
                                     color="red", markersize=8, linestyle="")
    def update(self, path, found, title):
        self.path_line.set_data([p[0] for p in path], [p[1] for p in path])
        self.found_marks.set_data([p[0] for p in found], [p[1] for p in found])
        self.robot.set_data([path[-1][0]], [path[-1][1]])
        self.ax.set_title(title)
            clear_output(wait=True)
            display(self.fig)
            def save(self, filename):
        self.fig.savefig(filename, dpi=150)

    def wait_close(self):
            plt.ioff()
            plt.show()

class NullVisualizer:
    closed = False

    def update(self, path, found, title):
        pass

    def save(self, filename):
        pass

    def wait_close(self):
        pass

def make_visualizer(grid, start, headless=False):
    if headless:
        return NullVisualizer()
        try:
        return PygameVisualizer(grid, start)
```

```
        except ImportError:
        print("pygame не встановлено - працюю через Matplotlib.")
        return MatplotlibVisualizer(grid, start)

def main(headless=False):
    if RANDOM_SEED is not None:
        random.seed(RANDOM_SEED)

    session_id, start, grid = start_session()
    print("Сесія:", session_id, "| старт:", start)

    viz = make_visualizer(grid, start, headless)

    current = start
    current_energy = math.inf

    path = [current]
    found = []
    cache = {}

    temperature_history = []
    k = 0
    no_improve = 0
    best_energy = math.inf

    steps_made = 0
    remaining = None
    last_signal = 0.0
    iteration = 0

    while iteration < MAX_ITERATIONS and steps_made < MAX_SERVER_STEPS:
        if remaining == 0 or viz.closed:
            break
        iteration += 1
        current_temperature = temperature(k)
        temperature_history.append(current_temperature)
        r = radius(current_temperature)
        candidate = neighbor(current, grid, r)
        if candidate is None:
            k += 1
            continue
        if candidate in cache:
            candidate_signal = cache[candidate]
        else:
            response = query_cell(session_id, candidate[0], candidate[1])
            candidate_signal = response["signal_level"]
            steps_made = response["steps_made"]
            remaining = response["remaining_foxes"]
            last_signal = candidate_signal
            if response["fox_caught"]:
                found.append(candidate)
                print(f"Знайдено об'єкт #{len(found)} у {candidate}, "
                      f"кроків: {steps_made}, залишилось: {remaining}")
                cache.clear()
                current = candidate
                path.append(current)
                current_energy = math.inf
                best_energy = math.inf
                k = 0
                no_improve = 0
                viz.update(path, found,
                           f"Кроків: {steps_made} | Залишилось: {remaining} | "
                           f"T: {T_START:.1f} (reheat)")
                continue
            cache[candidate] = candidate_signal
```

```
                if steps_made % SHOW_EVERY == 0:
                viz.update(path, found,
                           f"Кроків: {steps_made} | Залишилось: {remaining} | "
                    f"Сигнал: {last_signal:.2f} | T: {current_temperature:.2f}")
        candidate_energy = energy(candidate_signal)
        delta_energy = candidate_energy - current_energy
        probability = acceptance_probability(delta_energy, current_temperature)
        if random.random() < probability:
            current = candidate
            current_energy = candidate_energy
            path.append(current)
        if candidate_energy < best_energy - 1e-9:
            best_energy = candidate_energy
            no_improve = 0
        else:
            no_improve += 1
            if no_improve >= STAGNATION_LIMIT:
                k = 0
                no_improve = 0
                continue
        k += 1
    print("Знайдено об'єктів:", len(found))
    print("Залишилось:", remaining)
    print("Кроків (запитів) до сервера:", steps_made)
    if headless:
        return remaining, steps_made
    viz.update(path, found,
               f"Фінал | Кроків: {steps_made} | Знайдено: {len(found)}")
    viz.save("result_map.png")
    print("Збережено: result_map.png")
    fig2, ax2 = plt.subplots(figsize=(9, 4))
    ax2.plot(temperature_history)
    ax2.set_xlabel("Номер ітерації")
    ax2.set_ylabel("Температура T")
    ax2.set_title("Зміна температури (стрибки вгору - reheating)")
    ax2.grid(True)
    fig2.savefig("temperature.png", dpi=150)
    print("Збережено: temperature.png")

def run_experiments():
    global T_START, COOLING_RATE
    saved = (T_START, COOLING_RATE)
    summary = []
    for alpha, t_start in EXPERIMENTS:
        COOLING_RATE = alpha
        T_START = t_start
        steps_list = []
        success = 0
        for _ in range(RUNS_PER_EXPERIMENT):
            remaining, steps = main(headless=True)
            steps_list.append(steps)
            if remaining == 0:
                success += 1
summary.append((alpha, t_start, sum(steps_list) / len(steps_list), success))
    T_START, COOLING_RATE = saved
    print("alpha | T_start | середня кількість кроків | успішних запусків")
    for alpha, t_start, average, success in summary:
        print(f"{alpha:.2f} | {t_start:5.1f} | {average:8.1f} |
{success}/{RUNS_PER_EXPERIMENT}")

if __name__ == "__main__":
    if RUN_EXPERIMENTS:
        run_experiments()
    else:
        main()
```

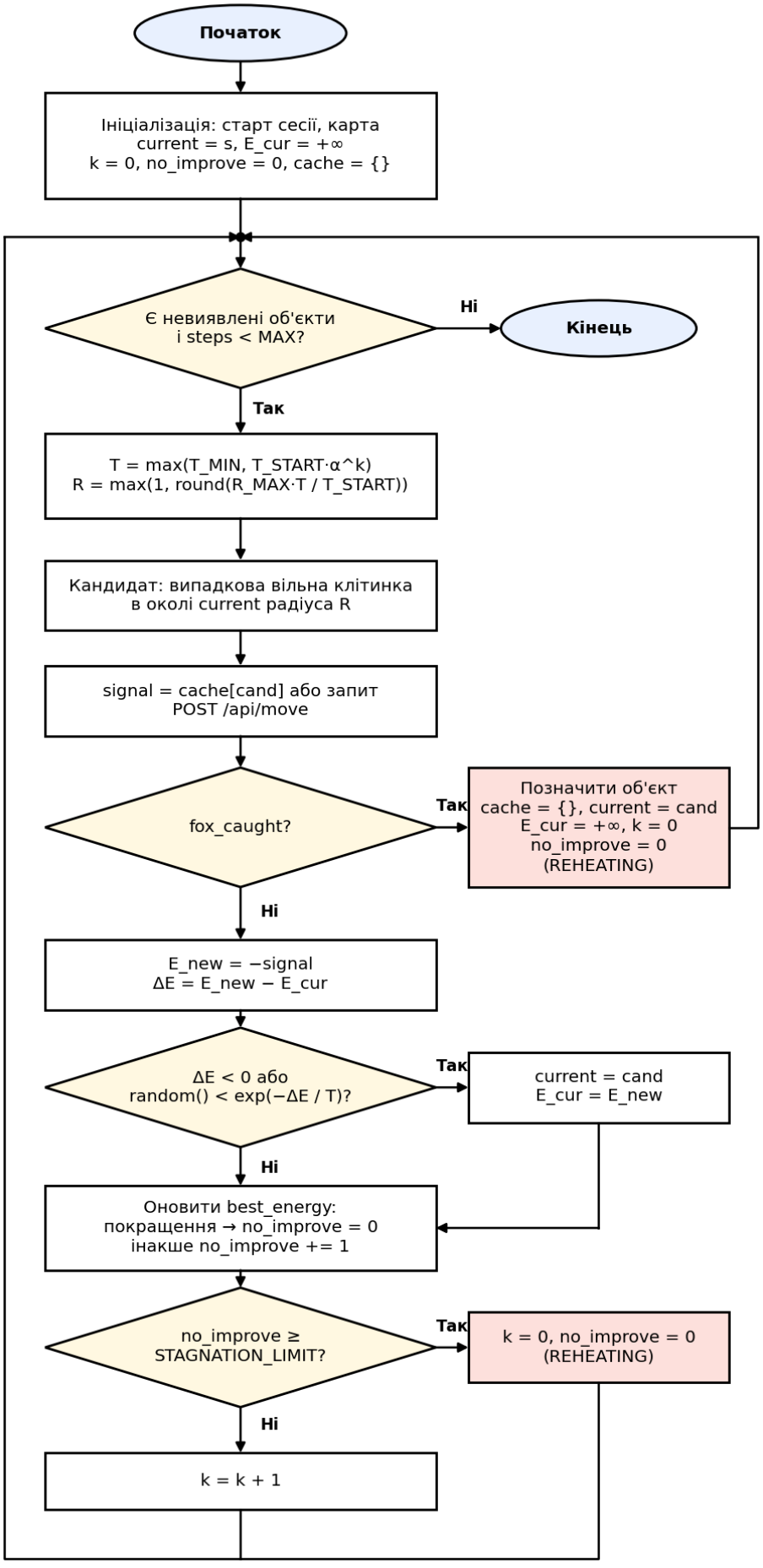


Рис. 1. Блок-схема алгоритму.

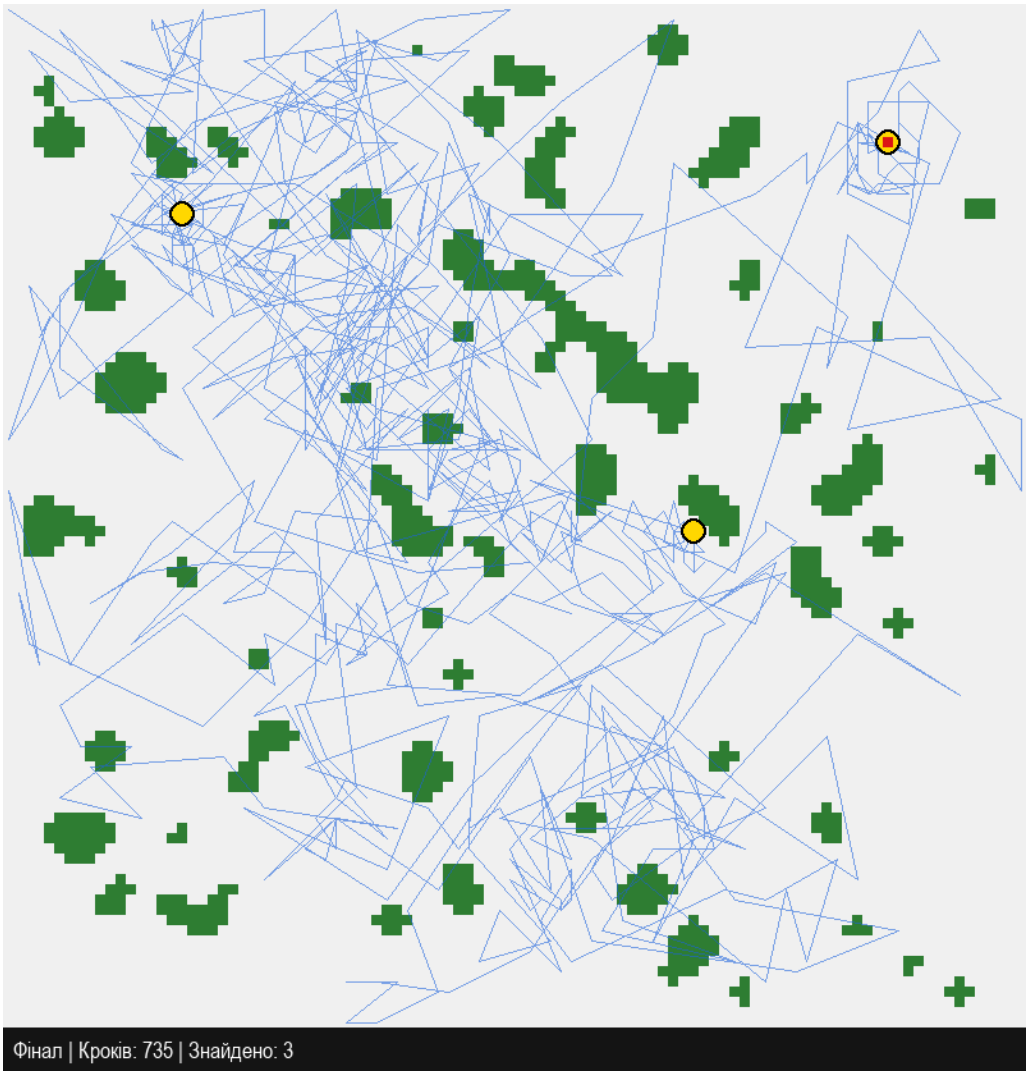


Рис. 2. Фінальне вікно з пройденим маршрутом.



Рис. 3. Зміна температури T від номера ітерації (стрибки вгору - reheating)

```
E:\обчисл\lab2\lab2_bubon\.venv\Scripts\python.exe E:\обчисл\lab2\lab2_bubon\main.py
pygame 2.6.1 (SDL 2.28.4, Python 3.13.1)
Hello from the pygame community. https://www.pygame.org/contribute.html
Сесія: 5ead45a8-889c-425a-988b-b16cf5050f09 | старт: (0, 0)
Знайдено об'єкт #1 у (67, 51), кроків: 191, залишилось: 2
Знайдено об'єкт #2 у (17, 20), кроків: 291, залишилось: 1
Знайдено об'єкт #3 у (86, 13), кроків: 735, залишилось: 0
Знайдено об'єктів: 3
Залишилось: 0
Кроків (запитів) до сервера: 735
Збережено: result_map.png
Збережено: temperature.png
```

Рис. 4. Результат виконання програми (консольний вивід)

Підсумкова кількість кроків: 735.

Знайдено об’єктів: 3 з 3.

Перший об’єкт знайдено в точці (67, 51) на 191-му кроці, другий - у точці (17, 20) на 291-му, третій - у точці (86, 13) на 735-му.

Для дослідження впливу параметрів алгоритм запускався з різними α та T_{start} . Кожна комбінація запускалася RUNS_PER_EXPERIMENT = 3 рази, у таблиці наведено середню кількість кроків.

α	T_{start}	Кроків у запусках 1 / 2 / 3	Середнє
0.90	30	257 / 233 / 891	460.3
0.95	30	425 / 192 / 151	256.0
0.98	30	1203 / 1384 / 496	1027.7
0.99	30	2676 / 698 / 2925	2099.7
0.98	10	371 / 409 / 583	454.3
0.98	50	331 / 2653 / 425	1136.3

Висновок: У ході роботи було реалізовано алгоритм імітації відпалу, котрий успішно знайшов усі приховані об’єкти на карті. По суті, алгоритм працює як шукач: він орієнтується на силу сигналу, аби знайти найкраще місце. На початку пошуку (коли температура висока), він може робити випадкові кроки навіть у гірші зони, оскільки це дозволяє не застрягти біля першого ж знайденого пагорба. Потім він поступово охолоджується і рухається лише туди, де сигнал кращий, аби точно визначити ціль.

Коли об’єкт знайдено, він зникає. Оскільки алгоритм на цей момент уже охолонув, застосовується стратегія повторного нагрівання - температура примусово скидається, себто алгоритму знову дозволяється робити великі випадкові кроки, аби знайти наступну ціль. Завдяки цьому всі 3 об’єкти було знайдено за 735 кроків. Якщо поглянути на результати експериментів, то найкраще себе показав коефіцієнт охолодження $\alpha = 0.95$, котрий дозволив знайти всі об'єкти в середньому лише за 256 кроків. Якщо ж взяти дуже повільне охолодження ($\alpha = 0.99$), алгоритм витрачає занадто багато часу на випадкові блукання - на це йде майже 2100 запитів. Більш швидке охолодження ($\alpha = 0.90$) працює непогано і забирає близько 460 кроків, але все ж поступається ідеальному балансу. Щодо початкової температури, то її рівень також суттєво впливає на час виконання. Зокрема, за однакового $\alpha = 0.98$ нижча стартова температура ($T = 10$) дала результат у 454 кроки, тоді як висока ($T = 50$) розтягнула процес аж до 1136 кроків. Оскільки за високих показників програма надто довго залишається активною і робить безліч непотрібних стрибків картою.
Себто, аби пошук працював максимально швидко, потрібно підібрати золоту середину. Програма не повинна бути гарячою надто довго (через високі стартові показники чи повільне охолодження), аби не витрачати кроки на марну біганину, але й не має охолоджуватися миттєво, щоб не застрягти далеко від цілі.